# ABSTRACT

Image data in traditional non-HDR cameras is represented in a linear dynamic range. Memory required for representation of linear dynamic range which would be perceptually equivalent to the dynamic range of human visual system is currently unavailable. Due to memory limitations process of capturing images is challenged by loss of data, which exceeds linear range of brightness and darkness. This report describes analysis and implementation of a selection of non-linear HDR algorithms to improve the perception of processed images within available resolution.

## Table of Contents

1. INTRODUCTION ..... 6

2. TONE MAPPING OPERATORS ..... 12

3. DESIGN METHOD ..... 15

4. PHOTOGRAPHIC TONE REPRODUCTION ..... 18

4.1. IMPLEMENTATION ..... 23

4.2. RESULTS ..... 25

5. TONE MAPPING FOR REALISTIC IMAGES ..... 29

5.1. IMPLEMENTATION ..... 31

5.2. RESULTS ..... 32

6. FAST BILATERAL FILTERING ..... 34

6.1. IMPLEMENTATION ..... 37

6.2. RESULTS ..... 39

7. DISCUSSION ..... 41

8. CONCLUSION ..... 44

9. SUGGESTIONS FOR FURTHER WORK ..... 45

10. BIBLIOGRAPHY ..... 46

11. APPENDIX A ..... 49

12. APPENDIX B ..... 51

# INTRODUCTION

Photography can be described as an art, science and process of creating images using a light-sensitive device.[1] Light can be captured either through a chemical process using a light sensitive material such as photographic film or electronically using an image sensor through a lens, which focuses light reflected or emitted by objects onto a light-sensitive surface inside the camera.[2] The amount of light entering the camera, defined as exposure, depends on the type of lens used and ISO sensitivity of the system and can be additionally controlled by adjustment of a shutter speed.[3]

While quality of captured photographs can be improved by careful adjustment of exposure through choosing appropriate sensor sensitivity, shutter speed and aperture of the lens, there are still significant obstacles in the way of obtaining photographs that faithfully represent real-life scenes.

Both film and digital image sensors have a range of sensitivity much lower than the human eye [3][4]. Dynamic range of image sensors covers at most 14 f-stops in comparison to the human visual system which is capable of recognizing up to 1,000,000 distinct light levels covering dynamic range of 20 f-stops. [5]

While digital cameras are capable of capturing high quality images in a lot of real-world scenes, sceneries combined of bright and low lighting conditions within the same frame still prove to be challenging for existing technology.

Film photography pushed the boundaries of capturing high dynamic range images by careful design of camera optics and material film [6]. However, while the number of chemically-based photography technologies is not expanding and majority of them are becoming obsolete, the diversity and availability of digital photography technologies is increasing. Although digital photography has become a foundation for many ground breaking inventions in photography and computer graphics making digital cameras a widely available consumer product, the new

technology introduced a completely new range of challenges and limitations entirely different from those faced in chemical photographic system design.[7] Nevertheless, despite immense differences between digital and film photography, both technologies use a linear function with limited exposure range to represent an image, introducing a bandwidth, which limits the dynamic range of brightness levels that can be captured in a photograph. As a result, captured images suffer from loss of information in the bright and dark areas outside the available range. All the areas in the image in which levels of light exceed the maximum level of brightness are clipped to that value and represented with white colour, regardless of its actual appearance. Such loss of detail is commonly referred to as over-exposure. Similarly, the areas of the image that are not illuminated enough to reach the lowest brightness level in the chosen exposure range are represented as black pixels and can be described as under-exposed. Under and over-exposed images are not desirable for two reasons. First of all, such loss of detail distorts faithful representation of the original scene. Secondly, it is impossible to extract any further information from the distorted parts of the image very often making it unsuitable to further processing.

High Dynamic Range Imaging is a branch of Digital Image Processing devoted to capture, storage, manipulation, transmission and display of images which would represent real-world scenes more faithfully.[6] There are a number of stages of HDR pipeline that are responsible for the quality of the images captured with digital cameras. The complexity and limitations of digital photography currently experienced can also be seen as areas for future improvement. First of all High Dynamic Range Capture depends on hardware specifications such as sensor resolution, quality and sensitivity as well as signal to noise ratio, photometric and spectral calibration. Moreover, quality of HDR capture is influenced by a memory available to store different levels of brightness and quantization factor, speed of the encoder, accuracy of the compression algorithms and available colour spaces. Currently, technology behind all the intermediate stages is reached the stage of development capable of capturing an HDR image. According to Holly Rushmeier[6] ‘HDR is finally ready to

enter the “mainstream” technology.’

One of the major obstacles still encountered in presenting HDR content is the limitation in contrast ratio available on the modern display devices. Even high end LCD televisions with a relatively high contrast ratio of 1:10,000 display 8bit per channel, limiting the colour and brightness shades to 255, which is not high enough for HDR content.[6]

Tone Mapping is one of the High Dynamic Range Imaging techniques that modify luminance and colours of already captured image to adapt its dynamic range to a lower dynamic range of a display.[6] This project explores the application of Tone Mapping Operators through the development of a filter program for testing effect of TMOs on different images. With increasing demand for HDR displays and high availability of processing power as opposed to brightness levels in available display devices Tone Mapping techniques offer a range of solutions and are becom an increasingly important area of High Dynamic Range Imaging.

Good understanding of requirements and effects of Tone Mapping can greatly influence the study of colorimetry and design of the colour spaces at the capture stage of digital photography pipeline.[7]

Filter program is an effective way of testing efficiency of different high dynamic range images captured in raw format widely available on DSLR consumer cameras. This approach addresses a demand in the industry for adjustment of the dynamic range of a displayed image without constructing a sophisticated and expensive hardware or designing and converting into independent colour spaces risking a distortion of the colour gamuts. An efficiently designed framework leaves a lot of space for further research and development of larger number of Tone Mapping Operators. With increasing accessibility of the graphic processors it is recommended that further research should be undertaken in the field on non-linear representation of dynamic range in cameras which could be later implemented at the capture stage. Object Oriented approach to creating Tone Mappers would allow combining and mixing different stages of already researched methods in search of

more effective and “universal tone mapper” and further investigation of tone mapper based on machine learning.

The first serious discussions on High Dynamic Range Imaging emerged in 1990s with introduction of first digital cameras. The first Tone Mapping Operator was introduced by Steve Mann in 1993[8], initiating research on dynamic range optimisation. At the same time Tumblin and Rushmeier proposed a Tone Mapping technique based on brightness reproduction inspired by Human Visual System called Realistic Image Compression[9], described in more detail later in this report. Only a year later Ward presented his research on logarithmic luminance mapping using a threshold-versus-intensity function [10]. Schlick approached Tone Mapping Operator by testing conventional mathematical functions used in digital signal processing and proposed a set of intuitive quantization techniques[11]. Ferweda et. al[12] in their paper entitled '*A Model of Visual Adaptation for Realistic Image Synthesis*' introduced a Tone Mapper, which simulated behaviour of Human Visual System and its ability to detect changes in threshold visibility, visual acuity as well as colour appearance and sensitivity over time. The model makes extensive use of psychophysical research on perception and uses mathematical models for rods and cones. This research inspired a brand new approach of perceptually based Tone Mappers.

A year later Larson et al.[13] introduced a modified version of a classic histogram equalization[2] which modelled colour sensitivity, glare and loss of acuity present in Human Visual System. This method turned out to be very commercially successful and has been implemented in a number of professional and customer photo-manipulation software packages.

Simultaneously to the progress in research and development of the above described global operators a number of studies were conducted in attempt to develop Tone Mapping techniques which are able to preserve local contrast. In 1993 Chiu et. al[14] introduced the first local operator called Spatially Non-uniform Scaling based on analysis of neighbouring pixels. Later research on local operators

include the perceptually based Multiscale Model of Adaptation and Spatial Vision[15] based on colour perception research distinguishing differences between long, middle and short wavelength responses of cones and rods. Photographic Tone Reproduction[16], implemented in HDR Simulator, and Tone Mapping Algorithm for High Contrast Images[17] uses a similar approach to preserving edges and avoiding halos[6].

A more advanced Retinex-based adaptive filter was proposed by Meylan et. al[18] in 2006 implementing parallel processing of luminance and chroma separately.

In 1999 Tumblin and Turk[19] introduced A Low Curvature Image Simplifier Operator based on preserving a local contrast by separating the large features from fine details by introducing a hierarchy filter separating high and low spatial frequencies of the image. Fast Bilateral Filtering[20], implemented in this project, was another example of frequency based Tone Mapping Operator. In 2002 Fattal et. al[21] proposed Gradient Domain Compression Algorithm modifying gradients based on assumption that high contrast appears in areas covered by large magnitude gradients.

A separate class of Tone Mappers based on image segmentation has been initiated by Yee and Pattanaik[22], who introduced Segmentation and Adaptive Assimilation for Detail-Preserving Tone Mapper which divides an HDR image into segments and calculates separate compressions for each region. This approach has been widely implemented in the popular JPEG image format[23].

Krawczyk et al.[24] introduced Lightness Perception operator based on an anchoring theory of lightness perception stating that the highest luminance value is perceived as white by the Human Visual System.

Lischinski et al.[25] introduced Interactive Local Manipulation which uses a set of brushes allowing the user to decide on where to change local luminace, chromacity and overexposure. This approach is common in photo-editing software.

Exposure Fusion[26], a technique popular among amateurs of photography, produces a Tone Mapped Image by merging a number of photographs from multiple captures.

New trends in Tone Mapping include colour correction[27], display adaptive Tone Mapping[28] and Tone Mapping for stereoscopic videos[29] and 3D-HDR imaged[30].

This report first gives a brief overview of theory behind Tone Mapping Operators. The next part deals with the design approach and hierarchy structures applied in the development of HDR Simulator. The third part describes theory, implementation and results achieved by applications of each of selected Tone Mapping Operators Individually. The final section presents analysis and critique of applied methodology including suggestions for further development. Appendices include a glossary of photography and digital image processing terms used in this report and code for the program.

# TONE MAPPING OPERATORS

Tone mapping can be defined as a method used in digital image processing and computer graphics to map contents of a High Dynamic Range image to a Lower Dynamic Range medium such as display or print.[6] Tone mapping addressed the problem of dynamic range reduction of an original photograph while preserving details such as colour appearance, local and global contrast in attempts to preserve faithful and realistic representation of the image. Different Tone Mapping Operators can have different objectives such as creating images most visually pleasing, images that mimic the experience of viewing a real scene and images that increase visibility of a detail. Tone Mapping can be employed in different fields of digital image processing ranging from medical and scientific imaging to traditional fine art photography. Each image can be uniquely categorized and processed in various ways depending on the requirements and as Holly Rushmeier writes: "Tone Mapping was found not to be a simple problem with just one simple solution but a whole family of problems."[6].

This project concentrates on algorithms that can be applied in consumer photography. The selected Tone Mappers attempt to reproduce pleasing images that match perception of the real-world scene. However, the framework of the program has been designed in a way that different it can be further extended to implement different applications of Tone Mappers.

Tone Mapping is performed using Tone Mapping Operators, which can be defined as:

$$f(I): \mathbb{R}_i^{w \times h \times c} \rightarrow \mathbb{D}_i^{w \times h \times c}$$

Where I is the image, w and h are width and height of the image and c is a number of colour bands; typically 3, because in most cases images are processed in RGB or YUV.

Usually only luminance is processed by TMO, whereas the colours are compressed using only linear quantisation. The reason for this is that Human Visual System is more sensitive to the difference is in brightness than the colour differences[6], therefore the defined above relationship

between the real-world and digital values of the image can be simplified to the following equation:

$$\begin{cases} L_d = f_L(L_W): \mathbb{R}_i^{w \times h \times c} \rightarrow [0,255] \\ \begin{bmatrix} R_d \\ G_d \\ B_d \end{bmatrix} = L_d \left( \frac{1}{L_W} \begin{bmatrix} R_W \\ G_W \\ B_W \end{bmatrix} \right)^s \end{cases}$$

Where s is a saturation factor that reduces saturation usually increased during tone mapping process, and $s \epsilon (0,1]$. Once the tone mapping operator f is applied the image is usually gamma corrected and colour channels are quantized and compressed to fit within the range of [0, 255].

Tone mapping operation greatly modifies colour gamut of the image, which can result in great difference in the process and colour correction has evolved into a separate branch of High Dynamic Range Imaging to tackle this problem.

Banterle et. al[6] classify Tone Mapping Operators into four main groups based on the algorithmic approach to the processing of the image. Tone Mapping Operators can be divided into global operators, local operators, segmentation operators and frequency and gradient operators. Global Operators modify image using identical functions to process each pixel. Local Operators use functions that consider not only the sole pixel but also its surroundings, referred to as pixel neighbourhood, which is the input of the Tone Mapping function. Segmentation Operators perform image segmentation prior to individual application of customized Tone Mappers to each segment of the image; and last but not the least Frequency and Gradient operator perform frequency filtering to separate gradients used to determine gradients before performing tone mapping on the low frequencies, keeping the high frequencies intact in order to preserve fine details.

Tone Mapping Operators can be also classified based on their design philosophy and intended objectives. Banterle et. al recognize three common design approaches, which include Perceptual Operators, which are based on models of Human Visual System and attempt to produce images

perceptually identical to how humans perceive real world scenes; Empirical Operators, which are inspired by photographic techniques; and Temporal Operators designed to compress HDR videos.

HDR Simulator provides three Tone Mapping Operators. First of them is a simple global operator called Realistic Image Compression[9], which is a perceptual operator introduced by Tumblin and Rushmeier in 1993 in their pioneering work on Tone Reproduction. HDR Simulator implements a version revised by Tumblin et. al in 1999. The second algorithm is a local empirical operator Photographic Tone Reproduction[16] based on common practices aiming to faithfully represent dynamic range in film photography. The last algorithm is Fast Bilateral Filtering proposed by Durand and Dorsey in 2002, which is an empirical frequency operator which consists of two stages and takes advantage of Realistic Image Compression.

This selection of algorithms was influenced by their ground breaking contribution to High Dynamic Range Imaging, variety of approaches to tone mapping including opportunity to present more sophisticated tone mappers, time restriction for the code implementation and mathematical complexity of the intermediate stages.

# DESIGN METHOD

HDR Simulator is a simple filter program for testing Tone Mapping operators to adjust dynamic range in photographic images. HDR simulator was developed using Python programming language.

Python programming language was chosen for variety of reasons. Python is a high-level programming language with emphasis on code readability which makes it easy to implement and debug as well as understand code already developed in it by other programmers.

Object-oriented programming, one of the features offered in Python, allows implementation of classes of objects, methods and inheritance, making the code more flexible for further re-implementation and development.[31]

Python offers rich libraries allowing fast development of scientific code. HDR Simulator makes extensive use of PyQt[32] graphical user interface library, NumPy and SciPy[33] mathematical packages as well as Python Image Library[34].

Above mentioned features strongly influence speed of development of the code, which was the major decision factor for choosing it as a tool to design and development of a mathematically complex image processing framework in within a short period of time.

HDR Simulator was designed heavily relying on the concept of object-orientation with intention of designing a framework with a great capacity for further development and extension.

HDR Simulator communicates with user through a simple graphic user interface which consists of a main window with a space for a processed image, a menu-bar loading, saving and adjusting the zoom factor of the displayed image and a toolbar with a drop-down menu offering a selection of Tone Mapping Operators that can be used to modify the dynamic range of the loaded image.

HDR class is an abstract mother class containing all the general information about the image

required for processing in with a Tone Mapper, such as location of the image, its height, width, pixel values, an Image object from Python Image Library, luminance and its debug log. The dynamic range optimisation is performed on images in raw data format *ppm*, supported by Python Image Library. HDR class contains a selection of methods which are universally used by different Tone Mapping Operators such as: obtaining luminance of the image from RGB colour scheme, obtaining its histogram, luminance mode, modification of luminance in RGB colour space, saving the image to a temporary location at any stage of processing, appending and saving debug log, a method for convolving and correlating two-dimensional arrays and converting pixel values from integer to floating point.

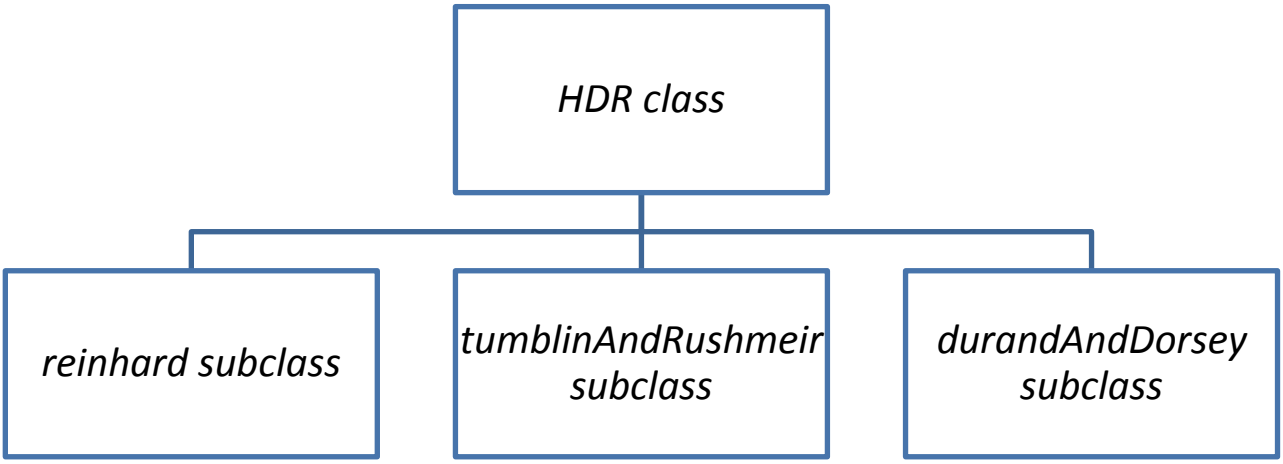



*Figure 1-Diagram illustrating relationship between mothe HDR class and TMO subclasses*

Each individual Tone Mapper is a subclass of the HDR class and inherits all the instances and methods of its mother class along with individually adding its own instances required for implementation of that specific algorithm as well as methods executed at different stages of its own image transformation process.

Once the user loads an image and selects the desired tone mapping operator, a pop-up window with a customized set of controls is displayed. When the user adjusts the parameters, their values are

returned to the GUI and function *loadParameters* creates an object of an appropriate subclass of HDR along with the parameters obtained from the operator class returned from the pop-up window. Then the *transform* method is invoked on the newly created subclass object. Each transform method for each of the Tone Mapping Operators varies greatly both in returned image, speed of execution as well as parameters required to execute it and will be described individually in the IMPLEMENTATION section of particular algorithm later in the report.

After the transformation of the image is complete, the resulting image is stored in a temporary location and the updated result is displayed within the main window of the program. The user now has an opportunity to save the resulting image in a desired location, repeat the process, perhaps choosing a different image or Tone Mapping Operator, zoom in the image to examine the details or quit the program.

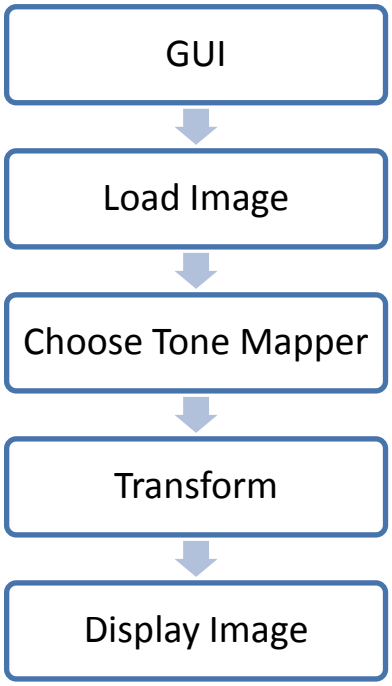



*Figure 2- Flowchart illustrating Image Transformation process in HDR Simulator*

# PHOTOGRAPHIC TONE REPRODUCTION

Photographic Tone Reproduction proposed by Reinhard et. al [16] introduced a method of mapping high dynamic range real world luminances to the low dynamic range using local operator techniques based on Zone System and dodging-and-burning film photography practices introduced by Ansel Adams[3],[35],[36].

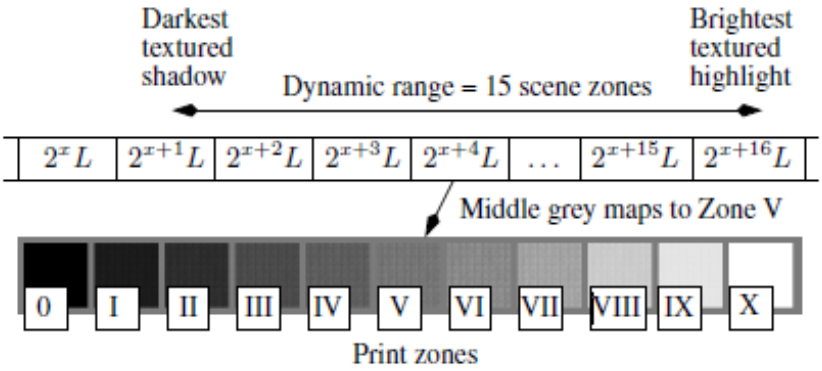



*Figure 3 - The mapping from scene zones to print zones. Scene zones at either extreme will map to pure black (zone 0) or white (zone X) if the dynamic range of the scene is eleven zones or more (Source: [16] Reinhard, E., Stark, Shirley, P. and Ferwerda, J., Photographic Tone Reproduction for Digital Images, ACM Transactions on Graphics, 21:3, 2002, 267-276)*

The Zone System is a technique simplifying the selection of correct exposure settings based on light metering and/or subjective assessment of the scene to produce the most realistic final print.[35]

The approximate luminance of the scene is divided into 11 zones, ranging from the pure black represented by the zone 0 to pure white represented by the zone X. Middle-grey is defined as a subjective average luminance of the scene and the key is used to indicate perceived brightness of the scene.[36] For instance a winter landscape full of snow would be high key and a dark interior of a bar would be a low key scene.



*Figure 4- Illustrates use of low(left) and high(right) key on the same scene. (Source: [16] Reinhard, E., Stark, Shirley, P. and Ferwerda, J., Photographic Tone Reproduction for Digital Images, ACM Transactions on Graphics, 21:3, 2002, 267-276)*

From the point of view of photography only the subjective measure of dynamic range is relevant to determine the zones as opposed to computer graphics, where the dynamic range is defined as the ratio of the highest and the lowest luminance of the scene. Zones are a logarithmic approximation of the scene luminances, therefore the dynamic range can be represented by a difference between the highest and the lowest perceived scene zones.[16]

Dodging-and-burning are film processing techniques aiming to equalize average luminance of the image and extract detail from hardly recognizable parts of an image. Dodging is a technique of restricting light at the bright regions of the image effectively increasing local contrast in these areas. Burning is a similar technique requiring application of extra light to increase the visibility of the dark regions of the photograph.[36]

Using the Zone System to assess the scene, the exposure parameters are set to faithfully represent the middle-grey of the scene. The photograph can realistically represent the captured scene, if the subjective luminance of the image is within a range of 9 zones of the dynamic range. In the parts of the image that exceed those 9 zones, the bright areas are mapped into white and dark areas are mapped into black, resulting in over- and underexposure.[35]

The process fully dependent on the subjective perception of the photographer is difficult to automate, however it still gives substantial control over the produced image. One of the main advantages of digital over film photography in this case is no loss of luminance information due to limitations of film processing and development. While this enables more accurate mapping of photographs, especially in their very bright and very dark regions, it also effectively requires more aggressive reduction of the dynamic range which is consecutively handled in dodging-and-burning part of the processing. The Photographic Tone Reproduction can be broken down into two stages. First, the luminance of the image is scaled, which is analogous to setting exposure in a camera, then consecutively dodging-and burning is applied. [16]

The method of application of Tone Mapping Reproduction can be summarized in the following steps. The first step is to approximate the key of the scene by calculating the logarithmic average of the scene luminance:

$$\overline{L_w} = \frac{1}{N}\exp(\sum_{x,y}\log(\delta + L_w(x,y)))$$

Where Lw(x,y) is the luminance value of pixel (x, y), N is the total number of pixels in the image and δ is a value added to bias the value of black pixels in the image.

Once the logarithmic average of the image is obtained, luminance of all the pixels in the image are scaled accordingly to the key chosen for the scene.

$$L(x,y) = \frac{a}{L_w}L_w(x,y)$$

where a is the key times* 0.18.

After key-scaling of the luminance, the local operator that mimics dodging-and-burning operation is applied over the whole region bounded by local contrast. To estimate the size of the local region Reinhard et al use a centre-surround function based on Blommaert's model for brightness perception [37]. The centre-surround function is constructed of circular Gaussian profiles defined as:

$$R_i(x,y,s) = \frac{1}{\pi(\alpha_i s)^2}\exp\left(-\frac{x^2+y^2}{(\alpha_i s)^2}\right)$$

Where s is a pre-defined range at which Gaussian profiles operate at each pixel of the image. The higher range s allows finer adjustment of the local contrast.

The image is analysed using such Gaussians producing a response *Vi* as a function of a pixel, scale s and luminance distribution L:

$$V_i(x,y,s) = L(x,y) \otimes R_i(x,y,s)$$

The resulting centre-surround function is defined as:

$$V(x,y,s) = \frac{V_1(x,y,s) - V_2(x,y,s)}{\frac{2^{\phi} a}{s^2} + V_1(x,y,s)}$$

Where $V_1$ and $V_2$ responses derived from $V_i$, using α1 = 1/2*$\sqrt{2}$ for $V_1$ and α2 = α1*1.6 for $V_2$.

$V(x,y,s)$ is therefore a difference of two Gaussians (centre and surround) at a certain radius for a given pixel and normalized by $\frac{2^{\phi} a}{s^2} + V_1(x,y,s)$. This response is then used to find the largest area of the pixels without sharp edges[6], the allowed contrast range is defined by a threshold $\epsilon$ to which each of the $V(x,y,s)$ is compared.

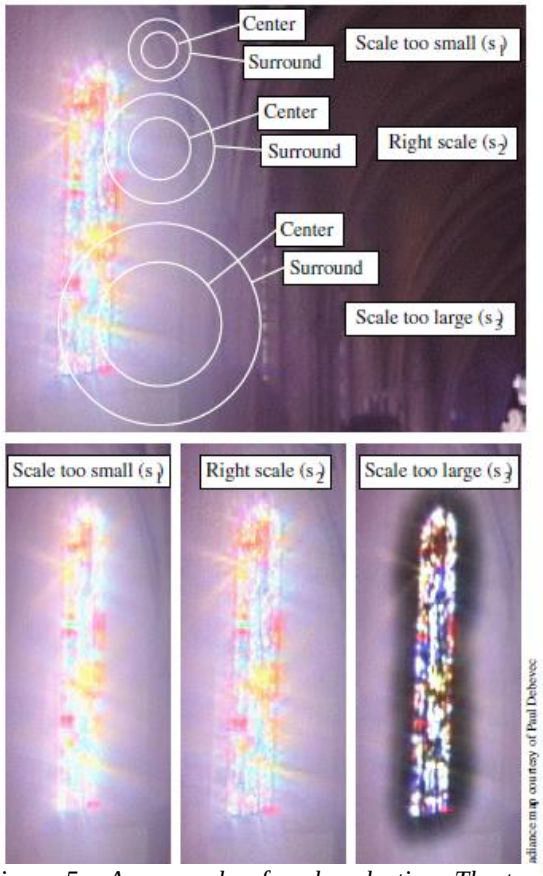



*Figure 5 - An example of scale selection. The top image shows center -An example of scale selection. The top image shows center and surround at different sizes. The lower images show the results of particular choices of scale selection. If scales are chosen too small, detail is lost. On the other hand, if scales are chosen too large, dark rings around luminance steps will form. [16] Reinhard, E., Stark, Shirley, P. and Ferwerda, J., Photographic Tone Reproduction for Digital Images, ACM Transactions on Graphics, 21:3, 2002, 267-276)*

$$|V(x,y,s_m)| < \epsilon$$

After the correct scale for each pixel is obtained, the luminance of the whole image is compressed using following equation:

$$L_d(x,y) = \frac{L(x,y)}{1 + V_1(x,y,s_m(x,y))}$$

This function performs local dodging-and-burning. A dark pixel in a bright region would satisfy threshold condition and the local contrast would remain unchanged, similarly a bright pixel in a dark region would also remain unchanged. Pixels that do not meet threshold condition

would be compressed, while the local contrast would remain intact. That results in a relative increase of a local contrast and is analogous to a pixel-by-pixel application of dodging-and-burning.

# IMPLEMENTATION

Photographic Tone Reproduction is handled within a subclass of *HDR* class called *reinhard*. Besides the inherited from HDR string *srcDirectory (*defining the source directory of the loaded image) it requires additional parameters, which include integers such as *key* defining the key of the photograph, *threshold* defining the threshold value of the image, *phi* which defines edge enhancing parameter, and *srange* which defines the maximum value of range s of pixels used to generate Gaussian profiles used to adjust the local operator.

Additional instances of the *reinhard* subclass include the above mentioned parameters as well as *minval*, defininining minimum luminance of the image, *maxval*, defining maximum luminance of the image and *drange*, defining dynamic range, obtained using method *getDynamicRange.*

Class *reinhard* allows default parameter setting by passing value 0 for all the integers in the constructor. In that case constructor calls a method *setDefault* and sets *key* using *setAutoKey* – automatic key selection method based on calculation of the average luminance value in the image; *threshold* value is set to 0.05; *phi* is set to 8 and *srange* is set to 2.

The *transform* function calls a set of functions handling intermediate stages of the process in the following order. *checkColourCoordinates()* checks whether the colour space is RGB. If the colour space is correct, then logarithmic average is obtained using *getLogAvLum().* Logarithmic average is scaled using *getScaledLuminace(logAvLum).* The V(s,x,y) and V1(s,x,y), described in the previous section are obtained using *getLocalContrast(scaledLuminance)*, which uses *getGaussianProfile* to create individual Gaussian profiles for *srange* different scales, convolves them with the scaled luminance of the image, compresses them and

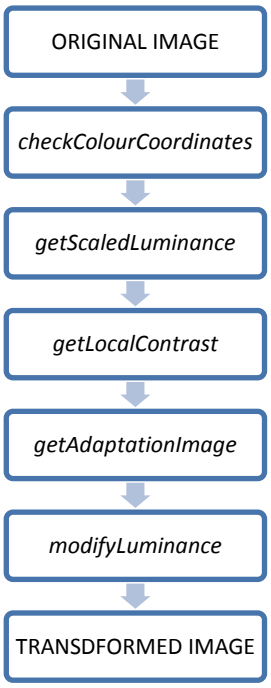



*Figure 6 - Flowchart illustrating transformation of an image with Photographic Tone Reproduction*

returning them as two arrays *Vs* and *V1*. *Vs* and *Vi* are fed to *getAdaptationImage*, which compares the scaled pixel values at each of *s* ranges with a product of *threshold* and the initial luminance value to determine which scale is most suitable and then if required transforms each pixel individually. Once the adaptation luminance is obtained the *transform* function calls *modifyLuminance* method inherited from the *HDR* class and updates the RGB values of the pixels. After completing the luminance modification the function returns the updated *reinhard* object as self.

The code developed for Photographic Tone Reproduction is attached in Appendix B

# RESULTS

Following images illustrate results obtained using Photographic Tone Reproduction. Time required for rendering images ranges from approximately 8mins for 128x128 pixels image with only 2pixel wide maximum Gaussian profile to 15h for the largest tested image 259x173pixels using 8pixel wide maximum Gaussian profile.



*Figure 7 - Effects of Photographic Tone Reproduction using different maximum radiuses of Gaussian profiles on outdoors scene.*
*A)original image*
*B) Max. Gaussian Profile of radius 2pixels,*
*C) Max. Gaussian Profile of radius 4pixels*
*D) Max. Gaussian Profile of radius 6pixels*
*E) Max. Gaussian Profile of radius 8 pixels*



*Figure 8 – Figure above represents effects of Photographic Tone Reproduction using different maximum radiuses of Gaussian profiles. The original image is a standard test image in image processing.*
*A)original image (Source: [2] Gonzalez, R.C, Woods, R.E., Digital Image Processing, Pearson, United Kingdom, 2007)*
*B) Max. Gaussian Profile of radius 2pixels,*
*C) Max. Gaussian Profile of radius 4pixels*
*D) Max. Gaussian Profile of radius 6pixels*
*E) Max. Gaussian Profile of radius 8 pixels*



*Figure 9 - Effects of Photographic Tone Reproduction using different maximum radiuses of Gaussian profiles on HDR interior scene next to a window*
*A)original image*
*B) Max. Gaussian Profile of radius 4pixels,*
*C) Max. Gaussian Profile of radius 6pixels*
*D) Max. Gaussian Profile of radius 8pixels*

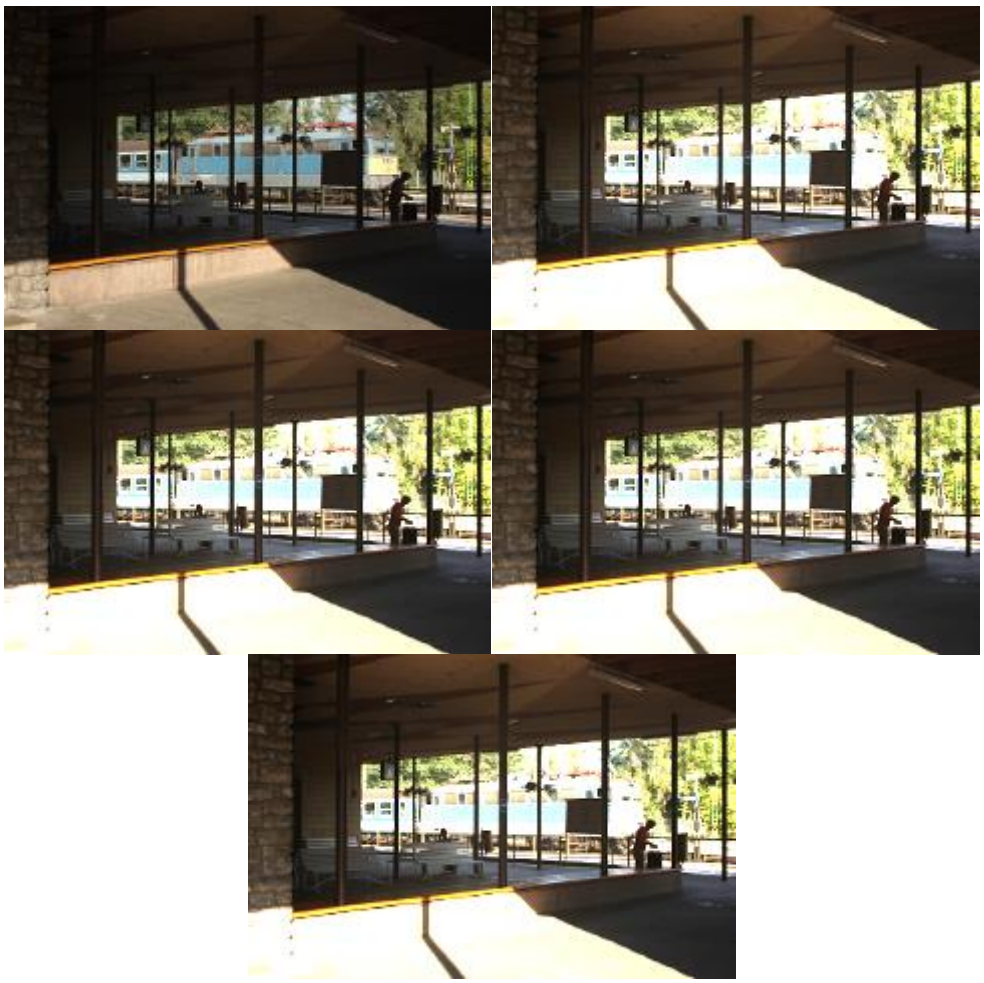


*Figure 10 - Effects of Photographic Tone Reproduction using different maximum radiuses of Gaussian profiles on high contrast outdoors scene. A)original image*
*B) Max. Gaussian Profile of radius 2pixels,*
*C) Max. Gaussian Profile of radius 4pixels*
*D) Max. Gaussian Profile of radius 6pixels*
*E) Max. Gaussian Profile of radius 8 pixels*



*Figure 11 - Effects of Photographic Tone Reproduction using different key values for a stander test image A)key 1 B) key 4 C) key 7 D) key 10*



*Figure 12 - Effects of Photographic Tone Reproduction using different maximum radiuses of Gaussian profiles on high contrast indoors scene. A)original image B) Max. Gaussian Profile of radius 2pixels, C) Max. Gaussian Profile of radius 4pixels D) Max. Gaussian Profile of radius 8pixels*

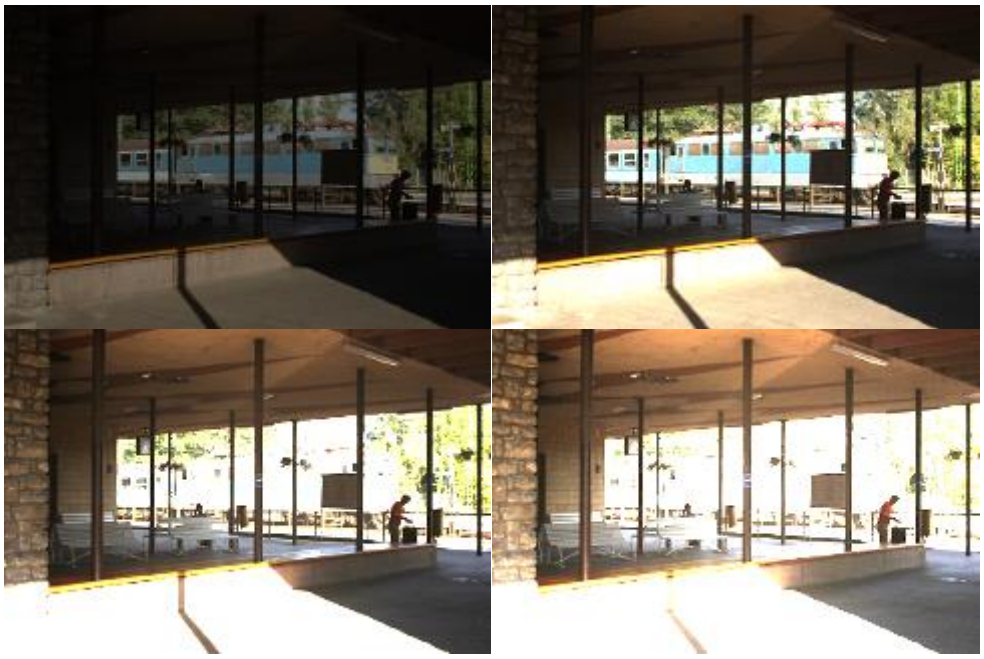


*Figure 13 - Effects of Photographic Tone Reproduction using different key values for a high contrast outdoors scene. A)key 1, B) key 3, C) key 6 D) key 10*

# TONE MAPPING FOR REALISTIC IMAGES

Tone Reproduction for Realistic Images introduced by Tumblin and Rushmeier[9] and later revised by Tumblin et al.[38] is a global operator preserving brightness of high-dynamic-range images was one of the first Tone Mapping Operators in the field of computer graphics.[6]

Tone Reproduction for Realistic Images is based on the Stevens and Stevens work on Human Visual System perception of brightness, radiance and introduced a Tone Mapping method simulating behaviour of fovea.[38]

Tumblin and Rushmeier introduced a global operator[9] defined as:

$$L_d(x) = mL_{da}\left(\frac{L_w(x)}{L_{w,H}}\right)$$

Where $L_{da}$ represents the adaptation luminance of the display (typically within the range of 30-100 cd/m2 for low dynamic range displays) and α is defined as:

$$\alpha = \frac{\gamma(L_{w,H})}{\gamma(L_{da})}$$

Where $\gamma$ is the contrast sensitivity function for a Human Visual System adapted to a luminance value x, defined by Stevens and Stevens [9] as:

$$\gamma(x) = \begin{cases} 1.855 + 0.4log_{10}(x + 2.3 * 10^{-5}) & for \;\; x \leq 100\; cd/m^2 \\ 2.655 & otherwise \end{cases}$$

Where parameter m is the adaptation-dependent scaling term, preventing anomalous grey night images and is defined as:

$$m = C_{max}^{\frac{\gamma_{wd}^{-1}}{2}}$$

Where $C_{max}$ is the maximum contrast of the display and $\gamma$ is defined as:

$$\gamma_{wd} = \frac{\gamma(L_{w,H})}{1.855 + 0.4log_{10}(L_{da})}$$

# IMPLEMENTATION

Realistic Image Compression is handled within a subclass of *HDR* class called *tumblinAndRushmeier*. Besides the inherited from HDR string *srcDirectory (*defining the source directory of the loaded image) it requires additional parameters, which include integers such as *Lda*, defined as adaptation luminance in [10,30] cd/m$^2$, *LdMax*, defined as maximum display luminance in [80-180] cd/m$^2$ and *Cmax*, defined as maximum contrast of the LDR monitor, typically [30-100] cd/m$^2$.

Additional instances of the *tumblinAndRushmeier* subclass include the above mentioned parameters and *lwa*, defining adaptation world luminance.

Class *tumblinAndRushmeier* allows default parameter setting by passing value 0 for all the integers in the constructor. In that case constructor calls a method *setDefault* and sets *Lda* value to 20, *Cmax* value to 80 and *LdMax* value to 30.

The *transform* uses function *gammaRushTMO* on *Lda* and *Lwa* to obtain values of *gamma_w* and *gamma_d* and the recursively calculated the new luminance of the image using the global Tone Mapping Operator defined in the previous section. After the new luminance is obtained the *transform* function calls *modifyLuminance* method inherited from the *HDR* class and updates the RGB values of the pixels. Once the luminance modification is completed the function returns the updated *tumblinAndRushmeier* object as self.

The code developed for Realistic TMO is attached in Appendix B

# RESULTS

Following images illustrate results obtained using Realistic Image Compression. Time required for rendering images takes approximately 2 minutes. Images were rendered using default parameters, that is *Lda* value was set to 20[cd/m$^2$], *Cmax* value was set to 80 [cd/m$^2$ ] and *LdMax* value to 30 [cd/m$^2$].



*Figure 14 - Effects of Realistic Image Compression using standard test image A)original image (Source: [2] Gonzalez, R.C, Woods, R.E., Digital Image Processing, Pearson, United Kingdom, 2007)*
*B)result of Realistic Image Compression*



*Figure 15 - Effects of Realistic Image Compression using outdoors scenery A)original image B)result of Realistic Image Compression*



*Figure 16 - Effects of Realistic Image Compression using a high contrast outdoors scenery A)original image*
*B)result of Realistic Image Compression*



*Figure 17 - Effects of Realistic Image Compression using a high contrast indoors scenery*
*A)original image*
*B)result of Realistic Image Compression*

# FAST BILATERAL FILTERING

Fast Bilateral Filtering is based in a decomposition of the processed image into a base layer and a detail layer.[20] This method modifies contrast only in the base layer, which is obtained using an edge-preserving filter. Fast Bilateral Filtering uses Bilateral Filter introduced by Tomasi et al.[39] to separates high and low frequency components from each other. Luminance compression is performed on the base layer, which contains the low frequency components of the luminance spectrum. Bilateral filter is non-linear and uses a Gaussian to compute weights of each pixel in the spatial domain, which is then multiplied by an influence function in the intensity domain. The role of the influence function is to reduce the weights of pixels with large intensity differences.

Additionally this method is executed using tone mapping framework designed by Durand and Dorsey.

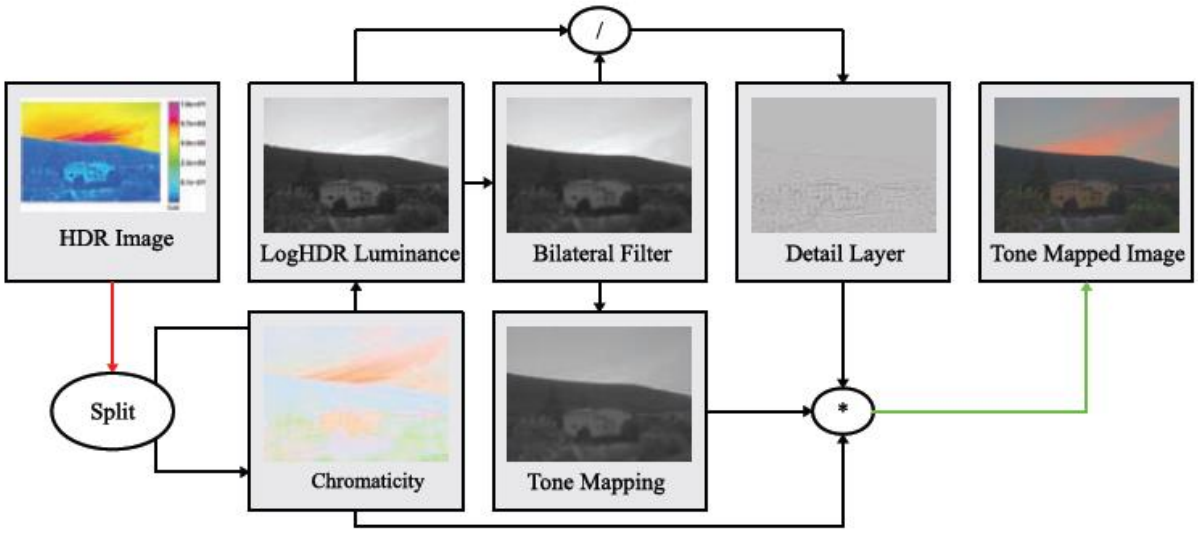



*Figure 18 - The pipeline of fast bilateral operator (Source: [6] Banterie, F., Artusi, A., Debattista, K. and Chalmers, A., Advanced High Dynamic Range Imaging, A K Peters, Ltd. Natick, MA, USA, 2011)*

Fast bilateral filtering is accelerated using a piecewise-linear approximation in the intensity domain and appropriate subsampling, which results in twice as fast code execution than a normal filter implementation. Fast Bilateral Filter requires no parameter setting.

Fast Bilateral Filtering is flexible in terms of choice of the tone mapper performed to compress the contrast in the base layer. This implementation uses Realistic Image Compression tone mapping operator[9] described in the previous section, which is the tone mapper suggested by the authors.[20] The work is done in the following steps. The luminance of the image is filtered using an edge-preserving filter. The base layer is compressed using Realistic Image Compression. The detail layer is merged with the compressed detail layer and the colour values are recalculated using new luminance of the image.

The output of the Bilateral Filter is the weighted average of the input. Each pixel of a spatial kernel f is filtered with a Gaussian filter and weighted accordingly to the intensity function g, which reduces the weight of those pixels with large differences in intensity. The Bilateral Filter is defined as:

$$I'(s) = \frac{1}{k(s)} \sum_{y \in \Omega} I(p) f(|s - p|) g(I(s) - I(p))$$

Where I is the image that is being filtered, s is a pixel of a spatial kernel at coordinates[x,y] and p is as a pixel within a Gaussian plane filtering the spatial kernel; k(x) is a normalization factor defined as:

$$k(x) = \sum_{y \in \Omega} f(|s - p|) g(I(s) - I(p))$$

The Gaussian intensity function is defined as:

$$g_\sigma(s) = \exp(-\frac{s^2}{2\sigma^2})$$

Where $\sigma$ is a parameter defining the threshold magnitude that the filter should stop. $\sigma$ is a product of two parameters, $\sigma_d$and $\sigma_r$:

$$\sigma = \sigma_d \sigma_r$$

where $\sigma_d$is a closeness parameter, defining the level of low-pass filtering performed with the Gaussian and $\sigma_r$ is a similarity parameter, measuring the photometric spread, responsible for mixing up of the pixels that are close together.[39]

Once a base layer is obtained with the use of bilateral filter, the detail layer is obtained by division of the original luminance of the image by the base layer, the base layer is filtered using Realistic Image Compression described in the previous section.

After the successful contrast reduction in the base layer the base layer and the detail layer are added back together and the image is reconstructed using new luminance value.

# IMPLEMENTATION

Fast Bilateral Filtering is handled within a subclass of *HDR* class called *durandAndDorsey*. Besides the inherited from HDR string *srcDirectory (*defining the source directory of the loaded image) it takes additional parameters, which include integers such as *Lda*, defined as adaptation luminance in [10,30] cd/m2, *LdMax*, defined as maximum display luminance in [80-180] cd/m2 for the Realistic Image Compression part of the processing.

*durandAndDorsey* subclass does not require any additional parameters and instances to be executed.

Identically to the previous *HDR* subclasses *durandAndDorsey* allows default parameter setting by passing value 0 for all the integers in the constructor. In that case constructor keeps these values as 0 and setting of the default values is handled by *tumblinAndRushmeier* itself, once it is called.

The *transform* function is executed in the following steps. Luminance of the image is obtained using *getLuminanceFromRGB.* After obtaining the luminance of the image Base Layer and detail layers are obtained using *getBilateralSeparation. getBilateralSeparation* sets *rangeS*, which defines the closeness parameter, described in the previous section to 4; and *spatialS*, defining the similarity parameter to 0.5. These values along with the luminance of the image are passed to the *bilateralFilter* function, which filters the luminance recursively using the Bilateral Filter function defined in the previous section and returns base layer of the image as its output. Detail Layer is obtained by dividing the luminance by the Base Layer. Once Base and Detail Layers are obtained by *getBilateralSeparation*, luminance values of the image are multiplied by the values of the Base Layer, pixels in the image are updated and the image is saved

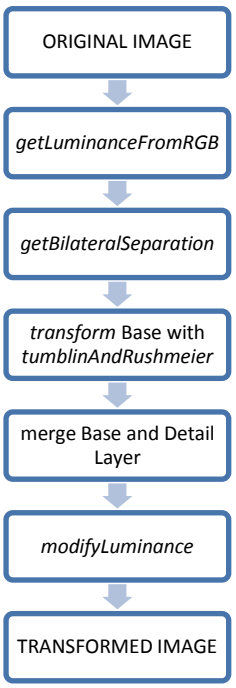



*Figure 7- Flowchart illustrating transformation of an image with Photographic Tone Reproduction*

to a temporary location. A new t*umblinAndRushmeier* object is created loading the Base Layer image from the temporary location and the Base Layer is transformed using Realistic Image Compression. The compressed image is saved again to a temporary location and the *srcDir, image, pixels, width, height* and *luminance* instances of the *durandAndDorsey* object are updated. Detail layer is added back to the luminance values of the image and the luminance values are updated by calling *modifyLuminance.* Once the luminance is modified the function returns the updated version of *durandAndDorsey* object as self.

The code developed for Fast Bilateral TMO is attached in Appendix B

# RESULTS

Following images illustrate results obtained using Fast Bilateral Filtering. Time required for rendering images takes approximately 20 minutes. Piecewise acceleration was not implemented. Images use defaults settings of Realistic Image Compression, that is *Lda* value was set to 20[cd/m$^2$], *Cmax* value was set to 80 [cd/m$^2$ ] and *LdMax* value to 30 [cd/m$^2$]. There is no noticeable difference between the effects of Fast Bilateral Filtering and Realistic Image Compression.



*Figure 19 - Effects of Fast Bilateral Filtering using standard test image A)original image (Source: [2] Gonzalez, R.C, Woods, R.E., Digital Image Processing, Pearson, United Kingdom, 2007)*
*B)result of Fast Bilateral Filterina*



*Figure 20 - Effects of Fast Bilateral Filtering using outdoors scenery A)original image*
*B)result of Fast Bilateral Filtering*



*Figure 21 - Effects of Fast Bilateral Filtering using a high contrast outdoors scenery A)original image*
*B)result of Fast Bilateral Filtering*



*Figure 21 - Effects of Fast Bilateral Filtering using high contrast indoors scenery*
*A)original image*
*B)result of Fast Bilateral Filtering*

# DISCUSSION

The aim of this project was to design and implement a filter program to enhance a high-bit-depth, full-colour image to optimise dynamic range of digital photographs. This report describes the hierarchical approach to the program design using object oriented programming to develop a framework for dynamic range optimisation using Tone Mapping techniques. The program demonstrates implementation of three different algorithms, each of them created using different techniques and design philosophies. The algorithms were chosen considering their mathematical complexity, contribution to High Dynamic Range Imaging, time required for their development and potential to demonstrate different approaches and methods used in Tone Mapping. Each of the algorithms can successfully transform an input image achieving different results and facing different limitations dependent on the design approach introduced by the authors.

Using Python made it possible to develop the framework and subclasses for three different Tone Mapping Operators and demonstrate functionality of the framework and its potential for introduction of much larger number of different operators, regardless of methodology they use and their mathematical complexity.

The high level of the language and automatic memory management, which make the development of the project straightforward and easy to debug is achieved by compromising the speed of execution of the code and higher memory requirements. PyQt is fully suitable for graphical user interface and Python Image Library proved to be very useful for accessing the image characteristics and properties. While SciPy provides a rich set of tools for complicated mathematical operations, which otherwise would take a lot of time to be developed and tested, the library is too slow for coding some of the more complex algorithms with more stages and repetitive iterations, making the code in its current shape unsuitable for the processing of high resolution images. Photographic Tone Reproduction is the algorithm which particularly suffers from severe decrease in speed of execution.

The main reason for that is the requirement for a larger radius Gaussian profile to analyze the high resolution images, as their contrast gradients have tendency to span across larger regions of pixels. For instance, a radius of 8 pixels might be entirely suitable for a low resolution image such as 256x256 pixels, while for a standard resolution image of 2048x2048 pixels the effect of local contrast adaptation would not be more noticeable than the effect of 1 pixel radius local operator for the previously mentioned low resolution image. That has a great impact on the speed of operation increasing the time of processing of high resolution not only proportionally to the number of pixels but also proportionally to the s range required to produce satisfactory effects. Therefore the speed of execution is compromised with accuracy of the processing and resolution of the image. In order to achieve time efficiency and exploit whole potential of Photographic Tone Reproduction, the mathematical operations would need to be managed by a library with much faster speed of access and execution, Gaussian profiles should be stored and generated in advance lookup tables and much more rigorous approach to implementation of nested loops and recursion should be employed.

While Photographic Image Compression gives much more satisfactory results than Fast Bilateral Filtering, not mentioning the Realistic Image Compression on its own, it requires much more computational power and more careful speed optimisation. Despite the fact that Bilateral Filtering is composed of two stages and unsuccessful implementation of the piece-wise acceleration due to the time constraints, Fast Bilateral Filtering turned out to be a substantially faster in execution than Photographic Tone Reproduction transformation of an image with a radius of Gaussian larger than 2pixels. An accurate implementation of piecewise acceleration could potentially speed up code execution of Fast Bilateral Filtering. Results of Fast Bilateral Filtering are disappointing. There is no noticeable difference between the results achieved with Realistic Image Compression. Both Tone Mappers return images of a worse quality than the input images, much darker than the original with no improved contrast. The Detail Layer is unrecognisable. The reason for that might be too small resolution of the test images or an unnoticed error in the code implementation. Results of Fast

Bilateral Filtering could be improved by providing a more sophisticated algorithm in the intermediate stage of processing instead of Realistic Image Compression.

The successful implementation of Photographic Tone Reproduction is very encouraging and suggests that by implementing more complex and carefully designed Tone Mapping Operator one might achieve much more perceptually satisfying results. The study also illustrates extreme importance of speed optimisation in Computer Graphics and proved that SciPy Python libraries are too slow for complex Digital Image Processing and High Dynamic Range Imaging.

# CONCLUSION

HDR Simulator is a simple software providing a selection of Tone Mapping Operators to optimise dynamic range of digital photographs. It is developed with emphasis on the class hierarchy and object orientation leaving a lot of space for further development and introduction of new different Tone Mapping Operators.

The program demonstrates the effects of three different algorithms: Photographic Tone Reproduction, Realistic Image Compression and Fast Bilateral Filtering. Each of the algorithms is based on different methodology and design approach.

Main weakness of HDR Simulator is lack of speed optimisation which was compromised due to the limited amount of time available for the project development. HDR Simulator was designed with consideration for the speed limitations by creating a structure flexible to modifications and improvements without the need for complete restructuring of the program.

The large capacity for implementation of various algorithms make HDR Simulator a useful tool for scientific simulations, however because of its speed limitations the program is not suitable for real-time image processing.

# SUGGESTIONS FOR FURTHER WORK

HDR Simulator was designed with intention of implementing a larger variety of Tone Mappers. Further work needs to be done to ensure quality assurance and improve speed optimisation. It would be interesting to make accurate timing measurements to determine exactly which parts of the code take the longest time to execute by using Python profiling or a time-stamp print outs in the log files. It is recommended to develop a custom library natively accessing a lower level language such as C++ or OpenGL libraries for time expensive calculations and array operations, which would allow use of powerful processing resources of a GPU.

The program would benefit from improving graphical user interface, especially by introducing an extra feature of an additional display window which would allow simultaneous comparison of the result of tone mapping with the image before processing.

# BIBLIOGRAPHY


[1] Spencer, D.A., *The Focal Dictionary of Photographic Technologies*, Prentice Hall, United Kingdom, 1973

[2] Gonzalez, R.C, Woods, R.E., *Digital Image Processing,* Pearson, United Kingdom, 2007

[3] Adams, A., The Camera, *Ansel Adams Photography Series,*Little, Brown and Company, 1980

[4] http://www.dxomark.com/Cameras/Ratings/%28type%29/usecase_landscape [accessed 04.04.2014]

[5] Klein, S.A., Carney, T., Barghout-Stein, L., Tyler, C.W., *Seven Models of Masking,* Proc. SPIE 3014, Human Vision and Electronic Imaging II, 13, 1997

[6] Banterie, F., Artusi, A., Debattista, K. and Chalmers, A., *Advanced High Dynamic Range Imaging*, A K Peters, Ltd. Natick, MA, USA, 2011

[7] Green, P. MacDonald, L., *Colour Engineering: Achieving Device Independent Colour,* John Wiley & Sons, Ltd., United Kingdom, 2003

[8] Mann, S., *Compositing Multiple Pictures of the Same Scene*, IS&Ts 46th Annual Conference, Cambridge Massachusetts, May 9-14, 1993

[9] Tumblin, J. and Rushmeier, H., *Tone Reproduction for Realistic Images*, IEEE Computer Graphics and Applications, 13:6, 1993, p 42-48

[10] Ward, G., *A Contrast-Based Scalefactor for Luminance Display*, Academic Press, Boston, MA, USA, 1994

[11] Schlick, C., *Quantization Techniques for Visualization of High Dynamic Range Pictures*, Proceedings of the Fifth Eurographics Workshop on Rendering, 1994, p7-18

[12] Ferweda, J.A., Pattanaik, N.S., Shirley P. and Greenberg, D.P., *A Model of Visual Adaptation for Realistic Image Synthesis*, SIGGRAPH '96: Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques, New York City, NY, USA: ACM, 1996, p249-258

[13] Larson Ward, G., Rushmeier, H. and Piatko, C., *A Visibility Matching Tone Reproduction Operator for High Dynamic Range Scenes*, IEEE Transactions on Visualization and Computer Graphics, 3:4, 1997, 291-306

[14] Chiu, K., Herf, M., Shirley, P., Swamy, S., Wang, C. and Zimmerman, K., *Spatially Nonuniform Scaling Functions for High Contrast Images*, Proceedings on Graphics Interface 93, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1993, p245-253

[15] Pattanaik, S.N., Ferwerda, J. A., Fairchild, M. D. and Greenberg, D.P., *A Multiscale Model of Adaptation and Spatial Vision for Realistic Image Display*, SIGGRAPH '98: Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques, ACM, New York City,

NY, USA, 1998

[16] Reinhard, E., Stark M., Shirley, P. and Ferwerda, J., *Photographic Tone Reproduction for Digital Images*, ACM Transactions on Graphics, 21:3, 2002, 267-276

[17] Ashikhmin, M., *A Tone Mapping Algorithm for High Contrast Images*, ERGW '02, Proceedings of the 13th Eurographics Workshop on Rendering, Eurographics Association, Aire-la-Ville, Switzerland, 2002, p145-156

[18] Meylan, L., Susstrunk, S., *High Dynamic Range Rendering with a Retinex-Based Adaptive Filter*, IEEE Transactions on Image Processing, 15:9, 2006, p2820-2830

[19] Tumblin, J. and Turk, G., *LCIS A Boundary Hierarchy for Detail-Preserving Contrast Reduction*, SIGGRAPH '99, Proceedings of the 26th Annual Conference on Graphics and Interactive Techniques, ACM Press/ Addison-Wesley Publishing Co., New York City, NY, USA, 1999, p83-90

[20] Durand, F. and Dorsey, J., *Fast Bilateral Filtering for the Display of High-Dynamic-Range Images*, ACM Transactions on Graphics, 21:3, 2002, p257-266

[21] Fattal, R., Lischinski D. and Werman, M., *Gradient Domain High Dynamic Range Compression*, ACM Transactions on Graphics, 21:3, 2002, 249-256

[22] Yee, H. Pattanaik, S.N. And Hughes, C. E., *Segmentation and Adaptive Assimilation for Detail-Preserving Display of High-Dynamic Range Images*, The Visual Computer, 19:7/8, 2003, p457-466

[23] http://www.jpeg.org/public/jfif.pdf

[24] Krawczyk. G., Myszkowski, K. and Seidel, H.P., *Lightness Perception in Tone Reproduction for High Dynamic Range Images*, The European Association for Computer Graphics 26th Annual Conference EUROGRAPHICS 2005, Blackwell, Dublin, Ireland, 2005, p635-645

[25] Lischinski, D., Farbman, Z., Uyttendaele, M. and Szeliski, R., *Interactive Local Adjustment of Tonal Values*, ACM Transactions on Graphics, 25:3, 2006, p646-653

[26] Mertens, T., Kautz, J. and Van Reeth, F., *Exposure Fusion*, PG '07, Proceedings of the 15th Pacific Conference on Computer Graphics and Applications, IEEE Computer Society, Washington, DC, USA, 2007, p382-390

[27] Mantiuk, R., Daly, S., Kerofsky, L., *Display Adaptive Tone Mapping* ACM Transanctions on Graphics 27:3 (2008), p1–10

[28]Mantiuk, R., Mantiuk, R., Tomaszweska, A., Heidrich, W., *Color Correction for Tone Mapping*, Proceedings of Eurographics 2009, 28:2, 2009, p193–202

[29] Du, S. P., Masia, B., Hu, S.M., Guttierez, *Ametric of visual comfort for stereoscopic motion*, ACM Transactions on Graphics, 32:6, New York, United States, 2013

[30]Selmanovic, E., Debattista, K., Bashford-Rodgers, T., Chalmers, A., *Generating Stereoscopic HDR images using HDR-LDR image pairs*, ACM Transactions on Applied Perception, 10:1, New York, United States, 2013

[31] https://www.python.org/ [accessed 09.04.2014]

[32] http://pyqt.sourceforge.net/Docs/PyQt4/ [accessed 09.04.2014]

[33] http://www.scipy.org/ [accessed 09.04.2014]

[34] http://effbot.org/imagingbook/pil-index.htm [accessed 09.04.2014]

[35] Adams, A., The Negative, *Ansel Adams Photography Series*,Little, Brown and Company, 1981

[36] Adams, A., The Print, *Ansel Adams Photography Series*,Little, Brown and Company, 1983

[37] Blommaert, F. J. J., Martens, J.B., *An Object-Oriented Model for Brightness Perception*, Spatial Vision 5, 1990, 1, 15–41.

[38] Tumblin, J., Hodgins, J. K., Guenter, B.K., *Two Methods for Display of High Contrast Images*, ACM Transactions on Graphics, 18:1, 1999, 56-94

[39] Tomasi, C., *Manduchi, R., Bilateral Filtering for Gray and Color Images*, Proceedings of the 1998 IEEE International Conference on Computer Vision, Bombay,India

[40] http://www.all-things-photography.com/digital-dictionary.html [accessed 12.04.2014]

# APPENDIX A - GLOSSARY OF PHOTOGRAPHIC TERMS

Excerpt from [40] http://www.all-things-photography.com/digital-dictionary.html:

**Aperture** – size of the lens opening that allows more controlling the amount of light falling onto the sensor formed by a diaphragm inside the actual lens.

**Colour Space** - Digital cameras use known colour profiles to generate their images. The most common is sRGB or Adobe RGB. This along with all of the other camera data is stored in the Exif header of the Jpeg file. The *colour space* information ensures that graphic programs and printers have a reference to the colour profile that the camera used at the time of taking the exposure.

**Compression** - A Digital photograph creates an image file that is enormous. To enable image files to become smaller and more manageable cameras employ some form of compression such as JPEG. RAW and TIFF files have no compression and take up more space.

**Contrast** - The measure of rate of change of brightness in an image.

**Depth of Field**- The range of items in focus in an image. This is controlled by the focal length and aperture opening of a lens. A large or wide aperture gives a shallow depth of field (not much range in focus) and a smaller or narrow aperture give a large depth of field (more range in focus).

**Dynamic Range** - This is a measurement of the accuracy of an image in colour or grey level. More bits of dynamic range results in much finer gradations being preserved.

**Exposure** - Amount of light that hits the image sensor of film controlled by the shutter speed and aperture.

**F-Stop** - Number indicating the size of the aperture. It is an inversely proportionate number as in F2.8 is a large opening and F16 is a small opening.

**Image Resolution** - This relates to the number of pixels per unit length of image. E.g. pixels per inch, pixels per millimetre, or pixels wide etc.

**Image Sensor** - Digital cameras use an electronic image sensor (CCD or CMOS), to gather the image data

**ISO** - The speed or light sensitivity of a captured image is rated by ISO numbers such as 100, 400, 800 etc. The higher the number, the more sensitive to light it is. Similar to film, the higher speeds usually bring on more electronic "noise" so the image gets grainier. An excellent program for cutting down this "noise" is Neat Image.

**Overexposure** - This is an image that appears much too bright. The highlights and colours are totally lost and usually unrecoverable even by top software. Either the shutter speed was too long or the aperture was too wide.

**Pixel** - The individual imaging element of a CCD or CMOS sensor, or the individual output point of a display device. This is what is meant by the figures 640x480, 800x600, 1024x768, 1280x960 etc., when dealing with the resolution of a particular digicam. Higher numbers are best.

**Render** - This is the final step of an image transformation or three-dimensional scene through which a new image is refreshed on the screen.

**Resolution** - The quality of any digital image, whether printed or displayed on a screen, depends on its resolution, or the number of pixels used to create the image. More, smaller pixels add detail and sharpen the edges.

**RGB** - (Red, Green and Blue). The primary colours from which all other colours are derived. The additive reproduction process mixes various amounts of red, green and blue to produce other colours. Combining one of these additive colours primary colours with another produces the additive secondary colours cyan, magenta and yellow. Combining all three produces white.

**Shutter Speed** - The time between pressing the shutter and actually capturing the image. This is due to the camera having to calculate the exposure, set the white balance and focus the lens. Is worse with smaller digicams whereas the better DSLR's now have little or no shutter lag, like the better film SLR's.

**Under exposure** - A picture which appears too dark because insufficient light was delivered to the imaging system. Opposite of over exposure.

# APPENDIX B - CODE

E:\HDRSimulator\src\hdr__init__.py 12 April 2014 22:55

```
'''
     Copyright (C) 2014  Katarzyna Terek

     This program is free software: you can redistribute it and/or modify
     it under the terms of the GNU General Public License as published by
     the Free Software Foundation, either version 3 of the License, or
     (at your option) any later version.

     This program is distributed in the hope that it will be useful,
     but WITHOUT ANY WARRANTY; without even the implied warranty of
     MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
     GNU General Public License for more details.

     You should have received a copy of the GNU General Public License
     along with this program.  If not, see <http://www.gnu.org/licenses/>.
'''

import Image
import os, os.path
import sys, string
import numpy as np
from copy import copy

try:
    import pylab
    pylab_loaded = 1
except: pylab_loaded = 0

class HDR:

    def __init__(self, srcDir):

        pathFolder = srcDir
        '''checking if the format is accepted
        accepted formats: PPM (extentions: ppm, pgm, pbm, pnm'''

        if (pathFolder[len(pathFolder)-1]=='p'):
            if ((pathFolder[len(pathFolder)-2]=='g')|
                (pathFolder[len(pathFolder)-2]=='b')|
                (pathFolder[len(pathFolder)-2]=='n')|
                (pathFolder[len(pathFolder)-2]=='p')):
                if(pathFolder[len(pathFolder)-3]=='p'):
                    self.imgFormat = 'ppm';
                else:
                    ##throw and exception 'Incorrect format'##
                    print 'Wrong format'

        self.srcDir = srcDir
        self.image = self.getImage(self.srcDir)
        self.pixels = self.image.load()
        self.width = self.image.size[0]
        self.height = self.image.size[1]
        self.luminance = self.getLuminanceFromRGB()
        self.log = ""
```

```
    def getImage(self, srcDir):

        image = Image.open(srcDir)
        return image

    def saveTemp(self):
        tempDir = os.environ['HDR']
        tempDir = tempDir + '/hdr'
        self.image.save(tempDir+'/temp.ppm')
        return tempDir+'/temp.ppm'

    def saveImage(self, path):
        self.image.save(path)

    def convertToFloat(self):

        for x in range(0, self.width):
            for y in range(0, self.height):
                pixel = list(self.pixels[x,y])
                pixel[0] = float(pixel[0])
                pixel[1] = float(pixel[1])
                pixel[2] = float(pixel[2])

                floatPixel = tuple(pixel)
                self.pixels[x,y] = floatPixel

    def appendLog(self, string):

        self.log = self.log + "\n" + string

    def saveLog(self):

        tempDir = os.environ['HDR']
        text_file = open(tempDir + "/log.txt", "w")
        text_file.write(self.log)

    def getHistogram(self):
        return self.image.histogram()

    def getLuminanceMode(self, histogram):

        mode = 0
        for x in range(0, len(histogram)):
            if (histogram[x]>mode):
                mode = x

        return mode

    def getLuminanceFromRGB(self):

        luminance = np.zeros(shape = (self.width,self.height))
        for x in range(0, self.width):
            for y in range(0, self.height):
                '''L = 0.27R + 0.67G + 0.06B'''
                L=0.27*self.pixels[x,y][0]+0.67*self.pixels[x,y][1]+0.06*self.pixels[x,
```

```
                y][2]
                luminance[x,y] = L

        return luminance

    def checkColorCoordinates(self):
        colourspace = self.image.getbands()
        if (colourspace != ('R', 'G', 'B')):
            print("Colour coordinates of the image should be in RGB colourspace.")
            return False
        return True

    def modifyLuminance(self, newLuminance):

        log = ""

        for x in range(0, self.width):
            for y in range(0, self.height):
                '''L = 0.27R + 0.67G + 0.2B'''
                pixel = list(self.pixels[x,y])
                pixel[0] = int(pixel[0]*(newLuminance[x,y]/self.luminance[x,y]))
                pixel[1] = int(pixel[1]*(newLuminance[x,y]/self.luminance[x,y]))
                pixel[2] = int(pixel[2]*(newLuminance[x,y]/self.luminance[x,y]))

                log = log + "pixel[0]: " + str(pixel[0]) + " newLuminance[x,y]: " + str
                (newLuminance[x,y]) + " self.luminance[x,y]: " + str(self.luminance[x,y
                ]) + "\n" +"pixel[1]: " + str(pixel[1]) + " newLuminance[x,y]: " + str(
                newLuminance[x,y]) + " self.luminance[x,y]: " + str(self.luminance[x,y
                ]) + "\n" + "pixel[2]: " + str(pixel[2]) + " newLuminance[x,y]: " + str
                (newLuminance[x,y]) + " self.luminance[x,y]: " + str(self.luminance[x,y
                ]) + "\n"

                updatedPixel = tuple(pixel)
                self.pixels[x,y] = updatedPixel

        self.appendLog(log)

    def convolve2d(self, array1, array2):

        width = np.shape(array1)[0]
        height = np.shape(array1)[1]

        fArray1 = np.reshape(array1, width*height, 1)
        fArray2 = np.reshape(array2, np.shape(array2)[0]*np.shape(array2)[1], 1)
        fconvolution = np.convolve(fArray1, fArray2, "same")
        a=0
        b=0

        convolution = np.zeros(shape=(width, height))
        for j in range(0, len(fconvolution)):
            if (a==width):
                a = 0
                b=b+1
                convolution[a,b] = fconvolution[j]
                a=a+1
```

```
        return convolution

    def correlate2d(self, array1, array2):

        width = np.shape(array1)[0]
        height = np.shape(array1)[1]

        fArray1 = np.reshape(array1, width*height, 1)
        fArray2 = np.reshape(array2, np.shape(array2)[0]*np.shape(array2)[1], 1)
        fcorrelation = np.correlate(fArray1, fArray2, "same")
        a=0
        b=0

        correlation = np.zeros(shape=(width, height))
        for j in range(0, len(fcorrelation)):
            if (a==width):
                a = 0
                b=b+1
                correlation[a,b] = fcorrelation[j]
                a=a+1

        return correlation
```

```
'''
     Copyright (C) 2014  Katarzyna Terek

     based on HDR_Toolbox by Francisco Banterle  - Copyright 2013

     This program is free software: you can redistribute it and/or modify
     it under the terms of the GNU General Public License as published by
     the Free Software Foundation, either version 3 of the License, or
     (at your option) any later version.

     This program is distributed in the hope that it will be useful,
     but WITHOUT ANY WARRANTY; without even the implied warranty of
     MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
     GNU General Public License for more details.

     You should have received a copy of the GNU General Public License
     along with this program.  If not, see <http://www.gnu.org/licenses/>.
'''


import numpy as np
import hdr

class reinhard(hdr.HDR):

    def __init__(self, srcDir, key, threshold, phi, srange, default):

        pathFolder = srcDir
        '''checking if the format is accepted
accepted formats: PPM (extentions: ppm, pgm, pbm, pnm'''

        if (pathFolder[len(pathFolder)-1]=='p'):
            if ((pathFolder[len(pathFolder)-2]=='g')|
                (pathFolder[len(pathFolder)-2]=='b')|
                (pathFolder[len(pathFolder)-2]=='n')|
                (pathFolder[len(pathFolder)-2]=='p')):
                if(pathFolder[len(pathFolder)-3]=='p'):
                    self.imgFormat = 'ppm';
                else:
                    '''throw and exception 'Incorrect format'''
                    print 'Wrong format'

        self.srcDir = srcDir
        self.image = self.getImage(self.srcDir)
        self.pixels = self.image.load()
        self.width = self.image.size[0]
        self.height = self.image.size[1]
        self.luminance = self.getLuminanceFromRGB()
        self.log = ""

        self.minval, self.maxval, self.dRange = self.getDynamicRange()

        if default:
            self.setDefault()
        else:
```

```
            if (key==0):
                '''automatic key selection'''
                self.key = self.setAutokey()
            else:
                '''key parameter 0.18 default'''
                self.key = key*0.09
            '''gamma correction 1.6 default'''
            self.gamma = 1.6
            '''threshold 0.05 default'''
            self.threshold = threshold
            '''phi factor'''
            self.phi = phi
            '''max range of local contrast adjustment'''
            self.srange = int(srange)


    def setDefault(self):
        '''sets default values'''
        self.key = 0.36
        self.gamma = 1.6
        self.threshold = 0.05
        self.phi = 8.0
        ''''size of the filter in pixels'''
        self.srange = 2

    def setAutokey(self):

        mode = self.getLuminanceMode(self.getHistogram())
        key = np.round((mode/self.maxval)/0.09)

        return key


    def getlogAvLum(self):

        '''Log - average luminance (approximate luminance of the scene,
used to tdetermine the key of the scene'''

        N = self.width * self.height
        sumTot = 0
        luminance = self.getLuminanceFromRGB()
        for x in range(0, (self.width)):
            for y in range(0, (self.height)):
                sumTot = sumTot + np.log(luminance[x,y]+0.01)

        logAvLum = np.exp(sumTot/N)
        self.appendLog("logAvLum: " + str(logAvLum))
        return logAvLum

    def getScaledLuminance(self, logAvLum):

        luminance = self.getLuminanceFromRGB()
        for x in range(0, (self.width)):
            for y in range(0, (self.height)):
                luminance[x,y] = (self.key/logAvLum)*luminance[x,y]
```

```
                self.appendLog("ScaledLuminance["+str(x)+"," +str(y)+ "]: " + str(
                luminance[x,y]))
        return luminance

    def lumBurnout(self, scaledLuminance, convRes1, sm):

        lumBurnout = np.zeros(shape=(self.width, self.height))
        for x in range(0, (self.width)):
            for y in range(0, (self.height)):
                lumBurnout[x,y] = scaledLuminance[x,y]/(1+convRes1[x, y, sm[x, y]])

        return lumBurnout

    def getGaussianProfile(self, s, alpha):

        '''creating a mesh of size 2*s*2+1
twice the size of s range + 1 for the origin'''
        xs = np.linspace(-s*2, s*2, s*2*2+1)
        ys = np.linspace(-s*2, s*2, s*2*2+1)
        vx, vy = np.meshgrid(xs, ys)

        Ri = np.zeros(shape=(s*2*2+1, s*2*2+1))
        for x in range(0, s*2*2+1):
            for y in range(0, s*2*2+1):
                Ri[x,y] = np.exp(-(np.power(vx[x,y],2) + np.power(vy[x,y],2)))/(np.
                power(alpha*s,2))/(np.power(np.pi*alpha*s,2))
        return Ri

    def getLocalContrast(self, luminance):

        alpha1 = 1/(2*np.sqrt(2))
        alpha2 = alpha1*1.6

        Vs = np.zeros(shape=(self.srange, self.width, self.height))
        Vis = np.zeros(shape=(self.srange, self.width, self.height))
        V1s = np.zeros(shape=(self.width, self.height))
        V2s = np.zeros(shape=(self.width, self.height))

        for s in range(1, self.srange):
            Rs1 = self.getGaussianProfile(s, alpha1)

            Rs1 = np.reshape(Rs1, len(Rs1)*len(Rs1[0]))
            luminance=np.reshape(luminance, self.width*self.height)
            '''center'''
            V1sflat = np.correlate(luminance,Rs1, "same")
            '''restructure V1s in 2D'''
            a=0
            b=0
            for i in range(0, len(V1sflat)):
                if (a==self.width):
                    a = 0
                    b=b+1
                V1s[a,b] = V1sflat[i]
                a=a+1
```

```
            Rs2 = self.getGaussianProfile(s, alpha2)
            Rs2 = np.reshape(Rs2, len(Rs2)*len(Rs2[0]))
            ''' surround'''
            V2sflat = np.correlate(luminance,Rs2, "same")
            '''restructure V2s in 2D'''
            a=0
            b=0

            for j in range(0, len(V1sflat)):
                if (a==self.width):
                    a = 0
                    b=b+1
                V2s[a,b] = V2sflat[j]
                a=a+1

            for x in range(0, (self.width)):
                for y in range(0, (self.height)):
                    self.appendLog("Vs[s,x,y] = ("+ str(V1s[x,y]) + "-" + str(V2s[x,y])
                     + ")/( "+ str(np.power(2,self.phi*self.key)) +"/(" + str(s^2) +
                    "+" + str(V1s[x,y])+")")
                    Vs[s,x,y] = (V1s[x,y] - V2s[x,y])/((np.power(2,self.phi*self.key
                    ))/(s^2) + V1s[x,y])
                    Vis[s,x,y] = V1s[x,y]

        return Vs, Vis

    def getAdaptationImage(self, luminance, V, V1, maximumLuminance):

        adaptationLuminance = luminance
        Vs = np.zeros(shape = (self.width, self.height))

        for s in range(0, (self.srange)):

            Vs = V[s]

            '''find indices of v higher than threshold'''
            for x in range(0, (self.width)):
                for y in range(0, (self.height)):
                    if (Vs[x,y]<self.threshold*luminance[x,y]):

                        adaptationLuminance[x,y] = maximumLuminance*luminance[x,y]/(1+
                        Vs[x,y])
                        self.appendLog("Adaptation Luminance [" + str(x) + "," + str(y)
                         + "]: "+str(adaptationLuminance[x,y]))

                        '''If the difference between the pixels for given s range for
                        local contrast
exceeds the threshold change the value, otherwise keep it'''

            '''after checking all the values returned modified luminance'''
            return adaptationLuminance

    def getCompressedRange(self,scaledLuminance, adaptationLuminance, maxLuminance):

        compressedLuminance = np.zeros(shape = (self.width, self.height))
```

```
        for x in range(0, (self.width)):
                for y in range(0, (self.height)):

                    compressedLuminance[x,y] = (scaledLuminance[x,y]*(1+scaledLuminance
                    [x,y]/np.power(maxLuminance,2))/(1+adaptationLuminance[x,y]))

        return compressedLuminance

    def getDynamicRange(self):

        minval = 1000;
        maxval = -1000;
        luminance = self.getLuminanceFromRGB()

        for x in range(0, (self.width)):
            for y in range(0, (self.height)):
                if ((luminance[x,y]<minval) & (luminance[x,y]>0.0)):
                    minval = luminance[x,y]
                if(luminance[x,y]>maxval):
                    maxval = luminance[x,y]
                    self.appendLog("New MaxValue: " + str(maxval))

        return minval, maxval, (minval/maxval)

    def transform(self):
        if (self.checkColorCoordinates()==False):
            print("Can't proceed with the algorithm execution, wrong colour
            coordinates")
            self.appendLog("Can't proceed with the algorithm execution, wrong colour
            coordinates")
        else:
            '''get dynamic range values'''
            print("MaxVal: " + str(self.maxval)+ "MinVal: "+ str(self.minval))
            self.appendLog("MaxVal: " + str(self.maxval)+ "MinVal: "+ str(self.minval))
            '''Logarithmic mean calculation'''
            logAvLum = self.getlogAvLum()
            print("Average luminance obtained.")
            self.appendLog("Average luminance obtained.")
            '''Luminance Scaling with Alpha and Average Logarithimic Luminance'''
            scaledLuminance = self.getScaledLuminance(logAvLum)
            print("Luminance scaled...")
            self.appendLog("Luminance scaled...")
            '''local contrast calculation'''
            Vs, V1 = self.getLocalContrast(scaledLuminance)
            print("Local contrast obtained.")
            self.appendLog("Local contrast obtained.")
            print("Calculating adaptation luminance...")
            self.appendLog("Calculating adaptation luminance...")
            adaptationLuminance = self.getAdaptationImage(scaledLuminance, Vs, V1, self
            .maxval)
            '''Range compression'''
            print("Range compression.")
            self.appendLog("Range compression.")
            '''Changing luminance'''
```

```
            print("Modifying luminance...")
            self.appendLog("Modifying luminance...")
            self.modifyLuminance(adaptationLuminance)
            print("Image Transformed.")
            self.appendLog("Image Transformed.")
            self.saveLog()
            return self
```

```
'''
     Copyright (C) Katarzyna Terek 2014 <katarzyna.terek@gmail.com>

    based on HDR_Toolbox by Francisco Banterle  - Copyright 2013


     This program is free software: you can redistribute it and/or modify
     it under the terms of the GNU General Public License as published by
     the Free Software Foundation, either version 3 of the License, or
     (at your option) any later version.

     This program is distributed in the hope that it will be useful,
     but WITHOUT ANY WARRANTY; without even the implied warranty of
     MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
     GNU General Public License for more details.

     You should have received a copy of the GNU General Public License
     along with this program.  If not, see <http://www.gnu.org/licenses/>.
'''

import numpy as np
import hdr

class tumblinAndRushmeier(hdr.HDR):

    def __init__(self, srcDir, Lda, LdMax, CMax, default):

        pathFolder = srcDir
        '''checking if the format is accepted
        accepted formats: PPM (extentions: ppm, pgm, pbm, pnm'''

        if (pathFolder[len(pathFolder)-1]=='p'):
            if ((pathFolder[len(pathFolder)-2]=='g')|
                (pathFolder[len(pathFolder)-2]=='b')|
                (pathFolder[len(pathFolder)-2]=='n')|
                (pathFolder[len(pathFolder)-2]=='p')):
                if(pathFolder[len(pathFolder)-3]=='p'):
                    self.imgFormat = 'ppm';
                else:
                    ##throw and exception 'Incorrect format'##
                    print 'Wrong format'

        self.srcDir = srcDir
        self.image = self.getImage(self.srcDir)
        self.pixels = self.image.load()
        self.width = self.image.size[0]
        self.height = self.image.size[1]
        self.luminance = self.getLuminanceFromRGB()
        self.log = ""

        if default:
            self.setDefault()

        else:
            '''adaptation display luminance in [10,30] cd/m^2'''
```

```
            self.lda = Lda
            '''maximum display luminance in [80-180] cd/m^2'''
            self.ldmax = LdMax
            '''maximum LDR monitor contrast typically between 30 to 100'''
            self.cmax = CMax
        '''adaptation world luminance cd/m^2'''
        self.lwa = np.exp(np.mean(np.reshape(np.log(self.luminance+2.3*1e-5), self.
        width*self.height, 1)))

    def setDefault(self):
        '''sets default values'''
        self.lda = 20
        self.cmax = 80
        self.ldmax = 30

    def gammaTumRushTMO(self, x):

        val=2.655
        if (x<=100):
            val=1.855+0.4*np.log10(x+2.3*1e-5)
        return val

    def transform(self):

        '''Range compression'''
        gamma_w = self.gammaTumRushTMO(self.lwa)
        gamma_d = self.gammaTumRushTMO(self.lda)

        gamma_wd = gamma_w/(1.855+0.4*np.log(self.lda))

        mLwa = np.zeros(shape=(self.width, self.height))

        Ld = np.zeros(shape=(self.width, self.height))
        for x in range(0, (self.width)):
            for y in range(0, (self.height)):
                mLwa[x,y] = np.power(np.sqrt(self.cmax),(gamma_wd-1))
                Ld[x,y]= self.lda*mLwa[x,y]*(np.power((self.luminance[x,y]/self.lwa),(
                gamma_w/gamma_d)))


        self.appendLog("Modifying luminance...")
        self.modifyLuminance(Ld)
        print("Image Transformed.")
        return self

    def getpixels(self):
        return self.pixels
```

```
'''
     Copyright (C) Katarzyna Terek 2014 <katarzyna.terek@gmail.com>

     Based on HDR_Toolbox by Francisco Banterle  - Copyright 2013

     This program is free software: you can redistribute it and/or modify
     it under the terms of the GNU General Public License as published by
     the Free Software Foundation, either version 3 of the License, or
     (at your option) any later version.

     This program is distributed in the hope that it will be useful,
     but WITHOUT ANY WARRANTY; without even the implied warranty of
     MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
     GNU General Public License for more details.

     You should have received a copy of the GNU General Public License
     along with this program.  If not, see <http://www.gnu.org/licenses/>.
'''

from PIL import Image
import numpy as np
import hdr
import realisticImages

class durandAndDorsey(hdr.HDR):

    def __init__(self, srcDir, lda, cmax, default):

        pathFolder = srcDir
        '''checking if the format is accepted
        accepted formats: PPM (extentions: ppm, pgm, pbm, pnm'''

        if (pathFolder[len(pathFolder)-1]=='p'):
            if ((pathFolder[len(pathFolder)-2]=='g')|
                (pathFolder[len(pathFolder)-2]=='b')|
                (pathFolder[len(pathFolder)-2]=='n')|
                (pathFolder[len(pathFolder)-2]=='p')):
                if(pathFolder[len(pathFolder)-3]=='p'):
                    self.imgFormat = 'ppm';
                else:
                    '''throw and exception 'Incorrect format'''
                    print 'Wrong format'

        self.srcDir = srcDir
        self.image = self.getImage(self.srcDir)
        self.pixels = self.image.load()
        self.width = self.image.size[0]
        self.height = self.image.size[1]
        self.log = ""
        if default:
            self.setDefault()
        else:
            self.lda = lda
            self.cmax = cmax
```

```
    def setDefault(self):
        '''sets default values'''
        self.lda = 80
        self.cmax = 100



    def bilateralFilter(self,luminance,sigma_d,sigma_r):
        ''' Pre-compute Gaussian domain weights.'''
        w=5
        xx = np.linspace(-w, w,(2*w+1))
        yy = np.linspace(-w, w,(2*w+1))
        X,Y = np.meshgrid(xx,yy)
        G = X

        for x in range(0, 2*w):
            for y in range(0, 2*w):
                G[x,y] = np.exp(-(np.power(X[x,y],2)+np.power(Y[x,y],2))/(2*np.power(
                sigma_d,2)))

        '''Rescale range variance (using maximum luminance).'''
        sigma_r = 100*sigma_r

        '''Apply bilateral filter.'''

        Base = np.zeros(shape = (self.width, self.height))

        for x in range(0, self.width):
            for y in range(0, self.height):
                ''' Extract local region.'''
                if ((x-w)<1):
                    iMin = 1
                else:
                    iMin = x-w
                if ((x+w)>self.width):
                    iMax = self.width
                else:
                    iMax = x+w
                if ((y-w)<1):
                    jMin = 1
                else:
                    jMin = y-w

                if ((y+w)>self.height):
                    jMax = self.height
                else:
                    jMax = y+w

                I = np.zeros(shape = (iMax-iMin,jMax-jMin))
                H = np.zeros(shape = (iMax-iMin,jMax-jMin))
                F = np.zeros(shape = (iMax-iMin,jMax-jMin))
                q = np.zeros(shape = (iMax-iMin,jMax-jMin))
                for i in range(0, (iMax-iMin)):
                    for j in range(0,(jMax-jMin)):
                        I[i,j] = luminance[i+iMin,j+jMin]
```

```
                    '''Compute Gaussian range weights'''
                    H[i,j] =  np.exp(-np.power((I[i,j]-luminance[x,y]),2)/(2*np.
                    power(sigma_r,2)))
                    '''calculate the bilateral filter response'''

                    for ix in range(iMin, iMax):
                        for jy in range(jMin, jMax):
                            F[i,j] = H[i,j]*G[ix-x+w,jy-y+w]
                    q = F[i,j]*I[i,j]
            Base[x,y] = np.sum(q)/np.sum(F)

    return Base


def BilateralSeparation(self,img):

    '''default parameters'''

    rangeS = 4
    spatialS = 0.5

    tmp = np.zeros(shape = (self.width, self.height))
    Base = np.zeros(shape = (self.width, self.height))
    Detail = np.zeros(shape = (self.width, self.height))

    for x in range(0, self.width):
        for y in range (0, self.height):
            tmp[x,y] = np.log(img[x,y]+1)

    Base = self.bilateralFilter(img,rangeS,spatialS)

    '''Extracting Base and Detail Layers'''
    '''Removing 0'''
    for x in range(0,self.width):
        for y in range(0, self.height):
            if (Base[x,y]<0):
                Base[x,y] = 0
                Detail[x,y] = self.maxLum
            else:
                Detail[x,y] = (img[x,y]/Base[x,y])

    return Base, Detail


def transform(self):

    if (self.checkColorCoordinates()==False):
        print("Can't proceed with the algorithm execution, wrong colour
        coordinates")
    else:

        lumChan = self.getLuminanceFromRGB()

        '''Fine details and base separation'''
        Lbase,Ldetail=self.BilateralSeparation(lumChan)
```

```
        '''tumblin-rushmeier TMO here'''


        for x in range(0,self.width):
            for y in range(0,self.height):
                lumChan[x,y]=lumChan[x,y]*Lbase[x,y]

        '''save img into a temporary location'''
        self.luminance = lumChan
        tempFile = self.saveTemp()

        Lda = 0
        LdMax = 0
        CMax = 0

        image = realisticImages.tumblinAndRushmeier(tempFile, Lda, LdMax,CMax, 1)
        tumblinImage = image.transform()
        tempFile = tumblinImage.saveTemp()

        '''Update image properties from the temporary file'''
        self.srcDir = tempFile
        self.image = self.getImage(self.srcDir)
        self.pixels = self.image.load()
        self.width = self.image.size[0]
        self.height = self.image.size[1]
        self.luminance = self.getLuminanceFromRGB()


        '''Adding details back'''
        for x in range(0,self.width):
            for y in range(0,self.height):
                print(str(self.luminance[x,y]))
                self.luminance[x,y] = self.luminance[x,y]*Ldetail[x,y]
        '''Modifying the image luminance'''
        self.modifyLuminance(self.luminance)
        print("Image Transformed.")
        return self
```